

BÀI 1: LẬP TRÌNH KẾT NỐI CSDL VỚI JDBC

Mục tiêu:

Cung cấp cho sinh viên kỹ năng lập trình cơ sở dữ liệu trong java, thực hiện kết nối và tương tác các hệ quản trị cơ sở dữ liệu khác nhau, sử dụng thành thạo các JDBC API giúp các ứng dụng Java có thể kết nối và thực hiện các thao tác truy vấn và xử lý dữ liệu.

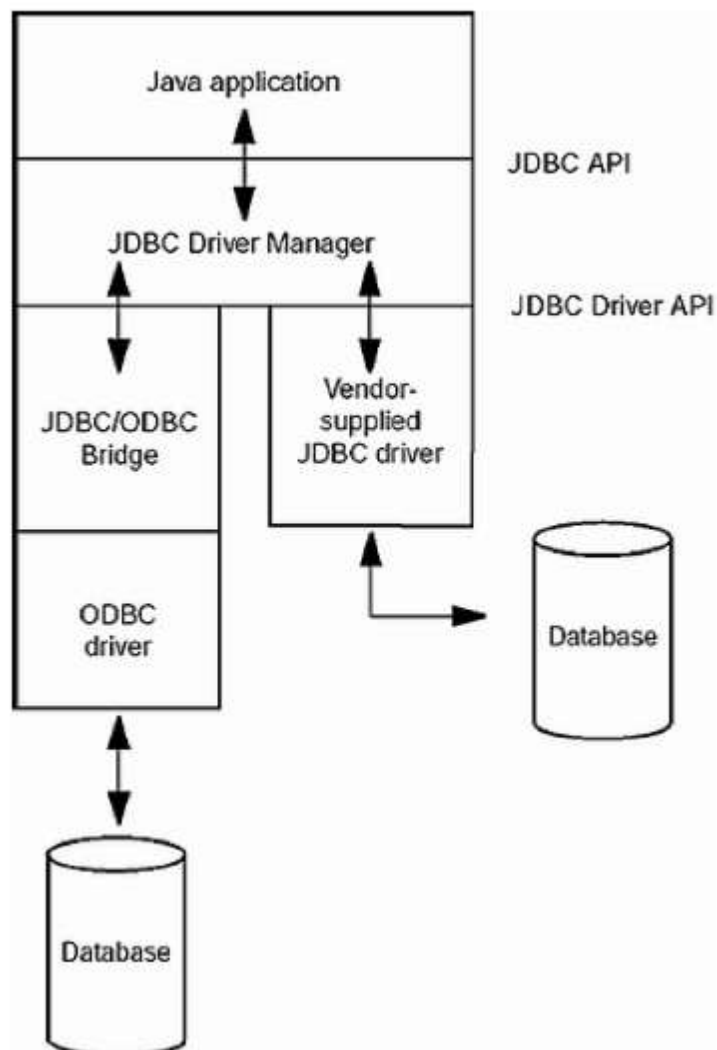
Nội dung:

- Kiến trúc JDBC
- Driver JDBC
- Kết nối và quản lý kết nối.

1.1. Kiến trúc jdbc

JDBC (Java Database Connectivity, java.sql.*) là một thư viện chuẩn dùng để truy cập dữ liệu quan hệ từ ứng dụng bằng ngôn ngữ truy vấn sql, một vài hệ cơ sở dữ liệu phổ biến như Access, MS SQL, Oracle, ...

Kiến trúc của JDBC gồm 2 tầng:

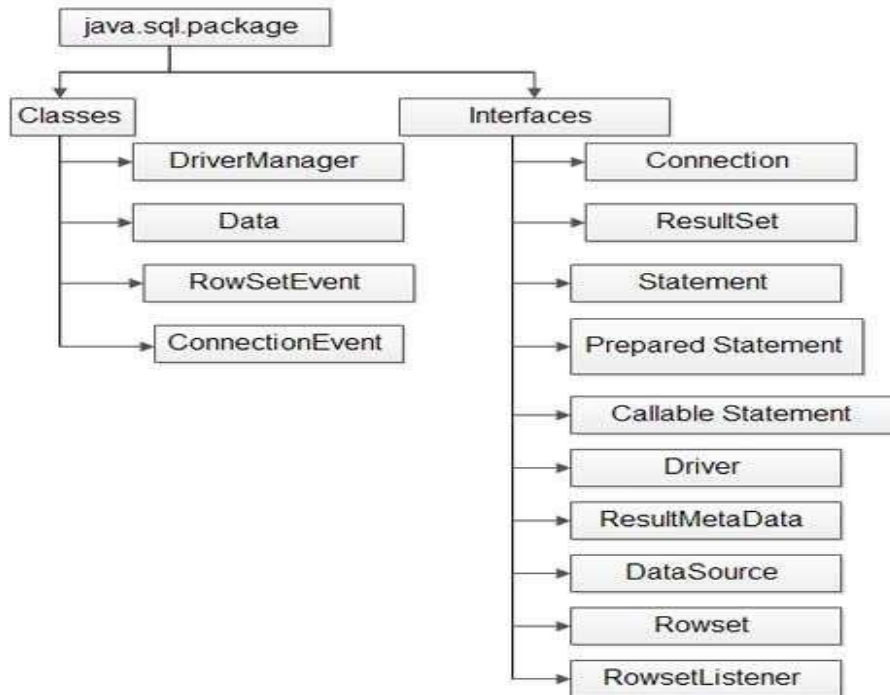


Hình 1: 1Kiến trúc JDBC

+ JDBC API: có nhiệm vụ chuyển các câu lệnh SQL cho bộ quản lý trình điều khiển JDBC

+ JDBC Driver API: thực hiện nhiệm vụ liên hệ với trình điều khiển của hệ quản trị cơ sở dữ liệu cụ thể.

- Các lớp và giao diện cơ bản trong JDBC API.



Hình 1: 2 Các lớp và giao diện cơ bản trong JDBC

1.2. Các trình điều khiển (jdbc driver)

Trình điều khiển JDBC là đoạn chương trình, do chính cầu hệ quản trị CSDL cụ thể thực hiện các câu lệnh SQL. nhà xây dựng hệ quản trị CSDL hoặc do nhà cung ứng thứ ba cung cấp, có khả năng yêu

Danh sách các trình điều khiển JDBC cho các hệ quản trị CSDL khác nhau được Sun cung cấp và cập nhật liên tục tại địa chỉ:

<http://industry.java.sun.com/products/jdbc/drivers.>

Các trình điều khiển JDBC được phân làm 04 loại khác nhau.

-Loại 1: **Bridge Driver** kết nối với các hệ CSDL thông qua cầu nối ODBC, dùng trong ngày đầu Java xuất hiện, ngày nay không còn phổ biến do có nhiều hạn chế. Nó cung cấp kèm trong bộ J2SE với tên: sun.jdbc.odbc.JdbcOdbcDriver.

-Loại 2: **Native API Driver**: chuyển các lời gọi của JDBC API sang thư viện hàm (API) tương ứng với từng hệ CSDL cụ thể. Trình điều khiển do nhà xây dựng hệ CSDL cung cấp.

-Loại 3: **JDBC-Net Driver**: chuyển các lời gọi JDBC API sang một dạng chuẩn độc lập với các hệ CSDL, và sau được chuyển sang lời gọi của hệ CSDL cụ thể bởi 1 chương trình trung gian. Trình điều khiển của các nhà cung ứng thứ 3 thường thuộc loại này. Lợi thế của trình điều khiển loại này là không cần cung cấp mã lệnh kèm theo và có thể sử dụng cùng một trình điều khiển để truy cập đến nhiều hệ CSDL khác nhau.

-Loại 4: **Native Protocol Driver**: chuyển các lời gọi JDBC API sang mã lệnh của hệ CSDL cụ thể. Đây là các trình điều khiển thuần Java, có nghĩa là không cần phải có mã lệnh của hệ CSDL cụ thể khi thi hành chương trình.

1.3. Viết chương trình làm việc với csdl qua jdbc

Với 1 chương trình viết bởi ngôn ngữ Java có sử dụng JDBC cho việc giao tiếp với cơ sở dữ liệu thì bạn phải thực hiện 5 bước như sau:

- 1) Nạp trình điều khiển kết nối (Load Driver)
- 2) Thiết lập kết nối đến cơ sở dữ liệu thông qua “Connection String” (Establish a database connection)
- 3) Tạo và thực thi 1 câu lệnh truy vấn đến cơ sở dữ liệu (Create and execute an SQL Statement)
- 4) Xử lý kết quả nhận về sau khi thi hành lệnh truy vấn thành công trên cơ sở dữ liệu (Process the results)
- 5) Đóng kết nối và giải phóng tài nguyên đã dành cho kết nối đó (Close the database connection)

Bước 1: Nạp trình điều khiển kết nối (Load Driver):

Để tiến hành nạp trình điều khiển kết nối, bạn phải dùng phương thức `forName` của lớp `Class` để tạo ra 1 thể hiện của lớp kết nối dựa trên giao thức mà JDBC cung cấp. Cú pháp của lệnh này như sau

```
public static Class forName(String className) throws ClassNotFoundException
```

Ví dụ : sử dụng kết nối theo kiểu 1 của JDBC (ODBC-JDBC Bridge) thì câu lệnh để load driver cho kiểu này là:

```
Class.forName("jdbc:odbc:JdbcOdbcDriver");
```

Chú ý: Chuỗi “`jdbc:odbc:JdbcOdbcDriver`” dùng làm tham số của hàm chính là giao thức điều khiển dành cho kết nối loại 1 của JDBC.

Trình điều khiển của MySQL:

```
Class.forName("org.gjt.mm.mysql.Driver");
```

Trình điều khiển của Oracle:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Trình điều khiển của Sybase: *Class.forName("com.sybase.jdbc.SybDriver");*

Trình điều khiển qua cầu nối ODBC:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

Chú ý: Nếu chúng ta có sử dụng tới OBBC, thì đầu tiên phải tạo nguồn ODBC cho ứng dụng.

Ví dụ trên windows (Access database), *Start -> Control Panel -> Administrative Tools -> Data Sources (ODBC)*

Đối với MySQL, ta phải tải một gói driver có tên MySQL Connector J JDBC driver về từ: <http://dev.mysql.com/downloads/> . Giải nén sau đó add tập tin .jar vào project, ví dụ trên Eclipse như sau:

- Trong trường hợp khác, bạn muốn kết nối đến cơ sở dữ liệu bằng kiểu kết nối thứ 4 của JDBC đối với cơ sở dữ liệu SQL Server, ta có thể thực hiện như sau

```
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
```

Chú ý : Trong tình huống này thì chuỗi

"com.microsoft.sqlserver.jdbc.SQLServerDriver" dùng làm tham số của

hàm chính là giao thức điều khiển dành cho kết nối loại 4 của JDBC đối với cơ sở dữ liệu SQL Server

Bước 2: Thiết lập kết nối đến cơ sở dữ liệu (Establish a database connection)

Việc kết nối với CSDL được thực hiện với một trong các phương thức sau trong lớp DriverManager:

-public static Connection getConnection(String url) throws SQLException: thực hiện kết nối với CSDL được yêu cầu. Bộ quản lý trình điều khiển sẽ tự động lựa chọn trình điều khiển phù hợp trong số các trình điều khiển đã được nạp.

-public static Connection getConnection(String url, String user, String pass) throws SQLException: tiến hành kết nối tới CSDL với tài khoản user và mật mã pass.

-public static Connection getConnection(String url, Properties info) throws SQLException: tương tự hai phương thức trên ngoài ra cung cấp thêm các thông tin qui định thuộc tính kết nối thông qua đối tượng của lớp **Properties**.

Kết quả trả về của các phương thức trên là đối tượng của lớp java.sql.Connection được dùng để đại diện cho kết nối đến CSDL.

Ví dụ: kết nối đến Database có tên qlCuocHen đã tạo ra trong SQL Server thông qua tài khoản sa với mật khẩu là 123456 thông qua kiểu kết nối thứ nhất của JDBC. Câu lệnh có thể viết như sau

```
String strConnect = "jdbc:odbc:driver={SQL Server};Server=BODUA-D778EAB2E;  
Database =qlCuocHen; UserName=sa; Password=123456";  
Connection cnObj = DriverManager.getConnection(strConnect);
```

Chú ý: Cú pháp của chuỗi kết nối được mô tả như sau

```
"jdbc:odbc:driver={SQLServer};Server=<ServerName>; Database=<YourDB>;UserN  
ame=<dbUser>;Password=<***>"
```

Tương tự, nếu sử dụng kiểu kết nối thứ nhất, bạn có thể sử dụng connection String theo cú pháp như sau

```
"dbc:sqlserver://<server_name>:<port>;databaseName=<YourDB>;user=<userName  
>; password=<pass>"
```

Bước 3: Tạo và thực thi 1 câu truy vấn dữ liệu dạng SQL (Create and Execute an SQL Statement)

Khi 1 kết nối đã được thiết lập đến Database thành công, để có thể tác động lên cơ sở dữ liệu thông qua connection đó thì bạn phải tạo ra 1 đối tượng để có thể thực thi các tác động lên cơ sở dữ liệu của chương trình thông qua các câu lệnh dạng SQL, đối tượng đó có thể là object của 1 trong 3 lớp sau :

- **Statement**
- **PreparedStatement**
- **CallableStatement**

(Trong phạm vi này, tôi mô tả cho việc sử dụng đối tượng Statement)

Như vậy, sau khi thiết lập kết nối thành công, ta phải tạo ra 1 đối tượng

Statement để nhận về 1 thể hiện của lớp này thông qua phương thức *createStatement()* của đối tượng **Connection** đã thiết lập.

Cú pháp

```
public Statement createStatement() throws SQLException  
Connection ketNoi = null;  
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
String Url="jdbc:odbc:driver={SQLServer};Server=BODUA-D778EAB2E; Databas  
e =qlCuocHen;UserName=sa; Password=123456";  
ketNoi= DriverManager.getConnection(Url);  
ResultSet kq=null;  
Statement stCmd = this.ketNoi.createStatement();
```

Bước 4: Xử lý kết quả nhận về sau khi thi hành lệnh truy vấn thành công trên cơ sở dữ liệu (Process the results)

Một đối tượng của lớp **Statement**, sau khi tạo ra có thể thực thi 1 lệnh SQL tác động lên cơ sở dữ liệu thông qua phương thức *executeQuery()* (hay *executeUpdate()* – thường dùng khi cần thực hiện các câu lệnh SQL dạng DDL: Data Definition Language). Kết quả trả về khi thực thi lệnh *executeQuery* sẽ là 1 đối tượng của lớp **ResultSet** chứa thông tin là kết quả truy vấn được từ cơ sở dữ liệu. Cú pháp của lệnh này như sau:

public ResultSet executeQuery(String strQuery) throws SQLException

Với 1 đối tượng **ResultSet**, bạn có thể sử dụng các phương thức tiêu biểu để thao tác đối với dữ liệu, hay điều khiển đối tượng này như sau

.next() – chuyển con trỏ sang mẫu tin kết tiếp để truy xuất dữ liệu

.getString(tenCot) – đọc thông tin của 1 trường thông tin dạng chuỗi có trong kết quả trả về của **ResultSet**. Tương tự đối với các kết quả dạng khác của các kiểu thuộc dạng Primitive data type như : int, boolean, byte, long, float, double ... Các phương thức tương ứng có thể lần lượt là : *.getInt(tenCot)*; *.getBoolean(tenCot)*; *.getByte(tenCot)*; *.getLong(tenCot)*; *.getFloat(tenCot)*; *.getDouble(tenCot)*... Các hàm này thường overload 2 dạng, dạng thứ nhất sử dụng tham số kiểu chuỗi chỉ ra tên của cột chứa thông tin muốn đọc, dạng thứ 2 kiểu int mô tả cho chỉ số của cột muốn đọc.

Ví Dụ: Giả sử sau khi đã tạo kết nối thành công, ta tạo 1 đối tượng của lớp **Statement** và đọc dữ liệu của bảng chứa thông tin là tên của các thành viên có trong **dsNguoiQuen** của database **qlCuocHen**, với các thông tin cần đọc về bao gồm các trường : **hoNQ**, **tenNQ**, **gioiTinh**, **tuoiNQ**. Ta sẽ làm như sau

```
Statement stCmd = ketNoi.createStatement();
String strQuery= "Select hoNQ, tenNQ, gioiTinh, tuoiNQ from dsNguoiQuen;";
ResultSet kq = stCmd.executeQuery(strQuery);
//-- Lặp trong khi còn đọc được dữ liệu trong ResultSet
While (kq.next()){
System.out.print(" Họ và tên :"+kq.getString("hoNQ") + " " +kq.getString("tenNQ");
System.out.print("Giới tính :"+(kq.getBoolean("gioiTinh") ? "Nam":"Nữ"));
System.out.println("Hiện nay : "+kq.getInt("tuoiNQ").toString()+" Tuổi");
}
Hoặc ta có thể truy xuất đến chỉ số của cột như sau
//-- Lặp trong khi còn đọc được dữ liệu trong ResultSet
```

```

While (kq.next()){
    System.out.print(" Họ và tên :"+kq.getString(1)+" "+kq.getString(2);
    System.out.print("Giới tính :"+(kq.getBoolean(3)?"Nam":"Nữ"));
    System.out.println("Hiện nay : "+kq.getInt(4).toString()+" Tuổi");
}

```

Bước 5: Đóng kết nối và giải phóng tài nguyên (Close the database connection)

```
public void close() throws SQLException
```

BÀI TẬP

Bài 1.

Tạo một cơ sở dữ liệu SinhVienDataBase và tạo bảng "students" với các trường như "id", "name", "age", và "grade" trên các hệ quản trị cơ sở dữ liệu khác nhau như MySQL, SQL server.

-Xây dựng các phương thức thực hiện khởi tạo đối tượng Connection theo như mẫu sau:

```

-public static Connection getMySQLConnection(String hostName, String dbName,
    String userName, String password) throws SQLException,
    ClassNotFoundException {...}

```

-Xây dựng phương thức sử dụng đối tượng **ResultSet** hiển thị thông tin của bảng ra *students*.

```
public void showData(ResultSet rs) throws SQLException{...}
```

Bài 2.

1. Xây dựng các phương thức thực hiện kết nối với cơ sở dữ liệu và thực hiện các phương thức thao tác (thêm mới, cập nhập, sửa, xóa...) với bản ghi trong bảng cơ sở dữ liệu.

```

public interface DAO<T> {
    public ArrayList<T> getAll(Connection con);
    public void insert(T ob,Connection con);
    public void updateC(T ob,Connection con) ;
    public boolean Delete(T ob,Connection con);
    public Object find(int id,Connection con);
    public Object findByName(String name,Connection con);
}

```

Bài 3.

Tổng hợp các kiểu ngoại lệ Exception và SQLException trong kết nối và truy vấn cơ sở dữ liệu.

Bài 4.

Xây dựng một ứng dụng đơn giản để quản lý giao dịch ngân hàng bằng Java, sử dụng JDBC và giao dịch để đảm bảo tính nhất quán trong cơ sở dữ liệu tương ứng với cơ sở dữ liệu như sau:

```
CREATE DATABASE banking;
USE banking;
CREATE TABLE accounts (
    account_number INT PRIMARY KEY,
    customer_name VARCHAR(255),
    balance DECIMAL(10, 2)
);
INSERT INTO accounts VALUES (1, 'John Doe', 1000.00);
INSERT INTO accounts VALUES (2, 'Jane Smith', 2000.00);
```

-Xây dựng phương thức để chuyển tiền từ tài khoản 1 sang tài khoản 2 sử dụng các khái niệm của Transaction và phương thức hiển thị số dư tài khoản.

